

多智能体协作下的叙事游戏工程实践

——以《这真的是对的时间线吗？》第一章为例

摘要

本文复盘一个真实交付完成的黑客松项目：知乎「跨次元游乐场」赛道的叙事游戏《这真的是对的时间线吗？》第一章。项目由一名人类队长 + 五个 AI 角色在 10 天内完成，产出三条发布通道（Web / macOS / Windows）、14 个叙事场景、三结局分支、完整美术与音频，以及一套可运行的验证体系。

本文不写“我们做成了什么”，而写“什么做法真的起了作用、什么做法看起来对但实际失效”。核心结论有三：

- 把平台机制转译为游戏机制，比把平台元素做成装饰更能站住产品逻辑；
- 数据驱动的契约层（人读 Markdown / 机器读 JSON）是让多个 AI 能并行工作而不互相破坏的前提；
- 可验证的证据链（探针、冒烟、契约校验）是多智能体协作中唯一可靠的质量控制手段——因为 AI 之间无法靠“看着像对”来互相验收。

文中同时给出 12 条可复刻实践与 5 条反模式，并论证该项目实施过程能否沉淀为 Skill 包。

关键词：多智能体协作；叙事游戏；数据驱动架构；工程验证；Godot

1. 引言：一个 48 小时项目的真实形态

黑客松项目的典型叙事是“48 小时极限冲刺”。本项目的真实形态不同：

- 日历跨度：2026-09-13 → 2026-09-22（10 天）
- 提交数：71 次
- 参与者：1 名人类（队长，负责拍板与最终验收）+ 5 个 AI 角色（各有明确定位，非“同一个 AI 多开”）
- 交付形态：不是 PPT 与录屏，而是三个平台的可运行构建 + 一个公开仓库 + 一套可重复执行的验证工具

这种形态带来一个传统黑客松没有的问题：多个 AI 同时在同一个仓库里工作时，如何保证彼此的工作不被覆盖、错误能被发现、结论能被复现？

本文以工程事实回答这个问题。

2. 项目概况与交付物

2.1 作品定位

玩家扮演一名因毕业选择而迷茫的大四学生，在阅读一篇没读完的知乎历史文章时被拉进文章构成的时间线。在 AI 向导“刘看山”的陪伴下穿越 1122 → 1127 → 1141 → 1142 → 1161 五个时间节点，通过观察、调

查、提问、验证面对同一个问题：

如果已经知道一个人的结局，我们是否真的有能力改变它？

2.2 交付清单（截至 2026-09-22）

维度	交付结果
发布通道	Web（GitHub Pages）、macOS 通用二进制、Windows 64 位单文件
版本	demo2-ch1-v0.2.4（公开 Release）；网页版在线可玩
叙事内容	14 个场景、9 个可调查对象、5 组刘看山三段式问答、3 种结局
系统能力	三层证据、条件路由、互斥取舍、存档读档、结算彩蛋
美术	10 张水墨背景、8 个角色（7 张立绘 + 刘看山三态动画）、4 件道具、13 项 UI、3 层特效
音频	4 段 BGM + 3 组环境循环 + 10 个交互音效（29 个母版）
工程	契约校验器、五路径冒烟测试、美术/音频探针、自动巡演录制、3 条 CI 流水线
文档	35 篇 Markdown（含初审产品说明计划书 v2.0）
资产治理	公开代码仓库 + 私有素材原件仓库（399 MB）

2.3 时间线

日期	阶段	关键事件
09-13	设计冻结与占位工程	时间线冻结、制作包交付、Godot 占位运行时建立并实跑
09-14	探索系统改版 + 资源接入	v0.2 契约交付与接入、美术/音频全量接入、多端导出、CDN 提速尝试与回退
09-16	平台接入与 UI 修复	知乎 OAuth 打通、面板可读性修复、v0.2.3
09-21	资产治理与元凶定位	补充素材发现、私密库建立、知识库清理、导出阻塞真因定位
09-22	收口	补充素材接入、v0.2.4 发布、首批美术需求全部交付完毕

3. 创作思路的理论框架

3.1 平台机制转译：把"提问有成本"做成机制

知乎内容的核心不是单向讲述，而是提问、回答、比较、追问。多数"平台 × 游戏"作品止步于把平台元素做成皮肤（用吉祥物、用配色）。本项目采用机制转译：把平台的运作规则直接变成玩家的决策约束。

平台机制	游戏机制	理论依据
提问需要组织语言与时机	刘看山验证预算仅 4 次	稀缺性驱动决策
问错问题浪费时间	缺道具时回答"信息不足"，但次数照扣	让失败模式即平台真实体验
收藏不等于读懂	拿到道具 ≠ 理解（见 3.2）	认知层次显式化
同一问题多种回答	三结局并存，系统不宣布唯一正解	保留判断权

关键设计：**insufficient**（信息不足）不是异常分支，而是主路径的一部分。玩家必须学会"先观察，再提问"——这正是平台使用者的真实成长曲线。

3.2 三层证据模型：把"知道"与"理解"拆开

这是第一章最核心的玩法结构，也是本项目最具复刻价值的原创设计：

层级	玩家动作	状态字段	语义
物证层	点击拾取家书 / 军书 / 路线图 / 证言	clues_found	证明事情发生过
行动资格层	找到传令牌、问清谁能执行	recall_recipient_found	允许尝试介入
解释层	带着道具向刘看山提问	shan_answer_recall / route_verified	把"看见"变成"理解"

三层齐备才解锁特殊结局。拿到军书却没问过"谁有能力执行"，玩家依然进不了完整介入链。

该模型可泛化为任意"知识型内容 → 互动体验"的产品的骨架：*发现 → 资格 → 理解*。

3.3 失败作为后果，而非重置

传统叙事游戏用"死亡重来"制造压力。本项目采用**后果制**：

- 漏拿关键物不 Game Over，而是立即给出**因果反馈**（"你拿到了军书，但不知道它该交给谁"）
- 玩家可带缺口继续，走一条明确说明缺口的"普通收束"
- 错过的物品会在结算时列出

理论依据：历史题材的悲剧性来自"不可撤销"，而不是"操作失误"。把失败写成后果，既服务主题，也降低探索的恐惧成本——鼓励玩家做高风险调查。

3.4 多结局的道德结构

三结局不做"好/坏"排序：

结局	语义
ending_canonical	历史轨迹：见证但不介入
ending_divergent	偏离但未改写：成功送达预警，但朝廷要一个答案的心没有改变
ending_truth	见证者真相线：保存普通人的证言

即便玩家打通完整介入链，结局也不是"岳飞活下来了"。这是对"历史不可被个人意志轻易改写"这一主题的机制化表达，也避免了把悲剧做成爽文。

3.5 可读性优先的视觉规范

美术接入后暴露出一个典型问题：**浅色宣纸底图 + 浅色文字 = 不可读**（对比度约 1.2:1）。修复过程沉淀出一条规范：

1. 面板必须有独立底板（不能直接压美术）
2. 底板透明度需与文字明度联合设计：浅底配深墨字、深底配浅字
3. 用 WCAG 4.5:1 作为可读性下限，并以像素统计做客观验收

最终方案：半透明宣纸底板（alpha 0.62）+ 近黑墨字，实测对比度 5.1 – 10.5:1（调查面板 / 刘看山面板 / 选项区 / 线索栏），且美术仍可透出（美术区平均亮度 184，接近纯美术）。

4. 技术架构

4.1 引擎与多端导出矩阵

项	选择	理由
引擎	Godot 4.3（4.3.stable.official.77dcf97d8）	2D 叙事表现力强、GDScript 迭代快、多端导出成熟
渲染	GL Compatibility	兼容性优先（Web 与老设备）
导出	Web(WASM) / macOS 通用 / Windows x64	一份工程三端产物
中文字体	Noto Sans SC 子集（702 字，8.3MB → 173KB）	不依赖系统字体回退（浏览器端关键）

关键决策：macOS 通用二进制需要启用 `import_etc2_astc`（Godot 对 arm64/universal 的硬性要求），而 Web 预设关闭该格式，因此 Web 包不受影响——多端导出必须按平台异构配置纹理格式。

4.2 内容与运行时分层

人读 Markdown，机器读 JSON。

载体	用途
Markdown	剧本、分镜、角色卡、线索卡、实现规格、史实审校
JSON（三层）	<code>state</code> （变量/条件/效果目录）、 <code>content</code> （对白/选项/调查对象）、 <code>manifest</code> （资源绑定/场景视觉/音频映射）

理论依据：AI 协作中，内容作者与代码作者需要不同的表示。Markdown 便于人审与 AI 生成，JSON 便于运行时消费与校验——同一事实的两种表示，边界清晰。

4.3 数据驱动运行时

运行时把契约当作可执行语义而非配置文本：

组件	能力
效果引擎	<code>set</code> / <code>increment</code> / <code>decrement</code> / <code>append_unique</code> / <code>guard</code> 守卫；支持 <code>effect:param</code> 参数替换
条件引擎	<code>all</code> / <code>any</code> / <code>eq</code> / <code>gte</code> / <code>lte</code> / <code>contains</code> ；命名条件递归求值
契约合并	v01 基础 + v02 补丁键级合并，未提及项保留

收益：剧本、数值、分支调整不需要改代码。这一点在多 AI 协作中价值极高——内容方（Codex）与实现方（DSH）可以并行工作，改动通过契约解耦。

4.4 交付链路工程

Web 端最大的敌人是首屏等待。本项目做了四层优化：

手段	效果
贴图按实际渲染尺寸重做（背景 1280×720、立绘 512×768）	资源包 14.2MB → 7.9MB
字体子集化（按全部文本用字，覆盖校验 100%）	8.3MB → 173KB
运行时未用资源排除出包（评审图、联络表图）	再省约 3MB
产物与文档分离（docs/ 排除出 pack）	避免截图被打进游戏包

最终：Web 资源包 8.0MB、引擎 wasm 33.7MB。

反模式警示：曾尝试用 CDN（jsDelivr）+ wasm 分片 + `<base>` 重定向提速，首屏从 12 分钟降到 20 秒，但因 CDN 分支缓存不随提交更新，导致新旧文件混用、游戏再也起不来，最终回退为自包含构建。详见 § 8.2。

4.5 平台接入：服务端中转模式

知乎开放平台 OAuth 接入采用服务端中转架构：

游戏 → `/api/oauth/start?handoff=<码>` → 知乎授权页 → `/auth/callback`
→ 服务端按 handoff 码发布结果 → 游戏轮询 `/api/oauth/handoff?code=<码>`

为什么不让游戏直连平台 API：OAuth 换 token 需要 App Key 与 Access Secret，这两个密钥绝不能进入浏览器代码。

为什么用轮询而非 `postMessage`：实测 `window.opener` 在弹窗被浏览器降级为新标签时为空，`postMessage` 不可靠；轮询方案同时适用于 Web 与桌面版，且不依赖跨域 cookie。

安全上：CORS 仅对白名单来源开放；未授权状态不缓存（否则授权后仍返回旧值）。

4.6 CI/CD 与制品发布

流水线	作用
<code>deploy-pages.yml</code>	装配站点（游戏 + Demo1 + 双卡彩蛋）并部署到 Pages
<code>build-oauth-image.yml</code>	构建 OAuth 服务镜像推送到 GHCR
<code>build-web-image.yml</code>	构建 nginx 静态站镜像（备用托管）

桌面产物通过 GitHub Release 发布，附未签名应用的打开指引（macOS 右键打开 / Windows SmartScreen）。

5. 多 AI 协作架构

5.1 角色与职责

角色	定位	核心产出
owner（王凯）	队长与总导演	范围取舍、五项拍板、最终验收
codex	叙事/系统	剧本、状态契约、线索、分支、实现规格
marvis	审校	史实复核、契约冲突、体验验收（分级打回）
dsh（本文主角）	桌面整合	Godot 工程、资源接入、构建运行、版本归档、跨工具衔接
super-zhendao（梁博森）	美术/UI	水墨资源、UI 组件、素材清单
kanshan（看山）	AI 向导内容	刘看山问答库文案、人设语气、检索适配

关键设计：角色不是"同一个模型多开"，而是能力与边界都不同。DSH 的边界明确写在文档里——不擅自改写剧情、时间线与角色设定，需要改动时交 Codex 与队长确认。

5.2 文件型协作接口

Codex 建立了一套异步文件消息协议（仓库 + Obsidian 双写）：

```
协作接口/  
├─ agent_chat_interface.md    # 协议：规则 + 角色地址 + 消息格式  
├─ inbox.md                  # 统一消息流（追加式，带 message_id）  
└─ task_board.md              # 任务板（owner / deliverable / status / evidence / next）
```

消息格式规定字段：`from / to / timestamp / type / priority / in_reply_to / needs_owner_decision / subject / evidence / next_action / status`。

价值：AI 之间不需要实时通道，消息落盘即是审计记录。任何结论都必须附 `evidence`（文件路径、commit、测试结果），这条规则直接淘汰了大量"我觉得应该没问题"式的交接。

5.3 审校闭环

Marvis 的审校采用分级 + 打回 + 复验制度：

级别	含义	处置
S1	史实错误，伤害可信度	必须改
S2	逻辑/契约冲突，导致运行时歧义	必须对齐
N	新引入的集成阻塞	按是否阻塞开工分级
P0/P1/P2	优先级	决定冻结与否

真实的对抗性案例：Marvis 曾判定"v0.2 不予冻结，D 包不具备开工条件，通过率 1.5/8"。DSH 的回应不是服从或争辩，而是逐条给出运行时实证：

- 复验所列"必错"项 (leave_effects 误记、字段无桥接、合并丢 nodes) 在运行时已用守卫与键级合并解决；
- 并用冒烟测试更正了审校方的数学判断：其"提问预算 3 < 4 不可达"的结论，经实测为*truth 线仅需 1 次验证、执行者可免费获得*，预算 3 成立。

方法论：审校方的价值在于发现问题，实现方的责任在于用可复现证据回应，而不是口头承诺。

5.4 唯一事实底稿

项目维护了一份《全量上下文与唯一事实底稿》，用于消除多 AI 之间的信息漂移。当不同角色的记录冲突时，以底稿为准，并同步回改所有文档。

这条看似行政的规定，实际上解决了多智能体最常见的问题：同一个事实在不同 AI 的记忆里有三个版本。

6. 工具与能力矩阵

6.1 模型路由

DSH 同时接入两条模型链路，按任务性质分配：

路由	协议与端点	典型用途
claude (<code>claude-fable-5</code> 等)	<code>anthropic-messages</code> / <code>api.openai-next.com</code>	长链路推理、架构设计、跨文件重构
kimi (<code>kimi-k3</code> / <code>kimi-k2.7-code</code>)	<code>openai-completions</code> / <code>api.openai-next.com/v1</code>	代码生成、批量文本处理

凭据存放于系统钥匙串或私有凭据文件，不入库、不入日志、不在对话中回显。

6.2 Skills（按需加载的能力包）

Skill	在本项目中的作用
<code>game-design-theory</code>	MDA 框架、玩家心理、平衡与成长曲线（用于三结局结构与预算设计）
<code>game-developer</code>	ECS、物理、对象池、状态机等工程模式
<code>game-engine</code>	Canvas/WebGL 渲染循环、碰撞、tilemap、音频
<code>game-feel</code>	屏震、顿帧、缓动、挤压拉伸、分层反馈（用于结算与铃印）
<code>game-ui-design</code> / <code>game-ui-ux</code>	HUD、菜单栈、锚点响应式、焦点导航、安全区
<code>threejs-game-ui-designer</code>	交互卡片的 3D 呈现
<code>zhihu-hackathon</code>	知乎开放平台接入（OAuth、CLI、接口规范）
<code>zhihu-search</code>	站内检索（用于内容素材与史实线索）
<code>ego-browser</code>	浏览器自动化（见 6.4）

观察：Skill 的价值不在"知道更多"，而在把领域约束前置。例如 `game-ui-ux` 的"焦点导航 / 安全区 / 事件驱动 HUD 更新"直接影响了 UI 层的实现方式。

6.3 MCP（模型上下文协议）

MCP	用途
Obsidian Local REST API	知识库读写：追加日志、更新进度表、维护协作接口、检索历史决策

MCP 在本项目的角色是"外部记忆"：AI 会话会结束，但知识与决策必须留在可检索的地方。项目的 154 个知识库文件（清理后 120 个）就是靠这一通道持续写入与回读。

6.4 浏览器自动化

`ego-browser` 承担了所有无法用 curl 验证的环节：

场景	做法
部署第三方平台	在 Sealos 控制台完成应用创建、注入环境变量、重启服务
验证 Web 产物	打开真实页面，抓取 canvas 像素判断是"渲染成功"还是"加载页/黑屏"
验证交互	按 1280×720 在画布内的等比缩放与黑边做坐标映射后点击，用画面差分确认状态推进
诊断前端故障	在页面上下文里执行 fetch/监听器探测，定位是"CSP 拦截"还是"跨域/会话"问题

方法论：像素统计与画面差分是"看不见画面"时的客观替代——前文所有可读性与渲染结论，都是这样得到的。

6.5 桌面与远程插件

插件	用途
<code>dsh-desktop</code> / <code>desktop-launcher</code> / 通知	宿主能力与任务完成提醒
<code>dsh-ssh</code> / <code>dsh-easyssh</code>	远程主机操作与远程工作区（本项目未深度使用，但在多机部署时是必要通道）
<code>dsh-aionui-panel</code>	右侧预览/文件/变更面板（代码审阅与差异查看）

6.6 自研验证工具（本项目最有复用价值的部分）

工具	作用	实证
<code>validate_content.py</code>	契约静态校验：场景引用、条件、效果、资源绑定、调查对象、问答绑定	通过时输出场景/效果/条件计数
<code>smoke_test.gd</code>	五条 QA 路径无头冒烟：三结局 + 漏线索兜底 + 提问耗尽 + 存档往返	5/5 + SAVE/LOAD OK
<code>probe_art.gd</code>	逐场景背景与逐说话人立绘覆盖探针	26/26 背景；军中同伴 512×768
<code>probe_audio.gd</code>	逐场景 BGM/环境与音效覆盖探针	10/10 音效

工具	作用	实证
qa_tour.gd	自动通关巡演 + 逐帧录制（配合 Movie Maker）	258 帧录制
prepare_cdn.mjs	产物后处理：wasm 分片 + 注入重组补丁 + CDN base 注入	见 § 8.2 的教训

这六个工具是整个项目最可迁移的资产——它们把"我认为改对了"变成"机器证明了改对了"。

7. 可复刻方法论：从项目到 Skill 包

7.1 十二条可复用实践

1. 契约先行：先定 `state/content/manifest` 三层 JSON，再写代码；内容与实现并行。
2. 人机双表示：Markdown 供人与 AI 审阅，JSON 供运行时消费，同一事实两处不冲突。
3. 数据驱动解释器：效果与条件用数据描述，代码只做解释——避免每加一个分支就改代码。
4. 键级合并契约：版本升级时未提及项保留，杜绝"补丁一合就丢场景"。
5. 验证工具先行于功能：探针与冒烟测试在接入资源之前就位。
6. 客观验收替代主观判断：像素对比度、画面差分、字节哈希、契约计数。
7. 失败即后果：不要用重置惩罚玩家，用后果服务主题。
8. 平台机制转译：复制平台的规则，而不是复制平台的皮肤。
9. 分层证据模型：发现 → 资格 → 理解，可泛化到任何知识型产品。
10. 服务端中转：任何涉及密钥的第三方接入都必须走服务端，客户端只收结果。
11. 消息落盘即审计：多 AI 协作的每条结论都必须附可复现证据。
12. 资产分层治理：代码进公开仓库、原件进私有仓库、构建产物不入库。

7.2 建议的 Skill 包结构

若要把本项目沉淀为可复刻的 Skill 包，建议如下组织：

```
narrative-game-dev/
├── SKILL.md                # 主入口：方法论 + 决策树
├── references/
│   ├── contract-layers.md  # 三层 JSON 契约设计与字段规范
│   ├── evidence-model.md   # 三层证据模型的泛化模板
│   ├── failure-as-consequence.md # 后果制设计模式
│   ├── multi-agent-collab.md # 角色划分、消息协议、审校闭环
│   ├── verifiability.md    # 探针/冒烟/像素验收的具体实现
│   ├── delivery-pipeline.md # 多端导出、体积优化、发布清单
│   └── pitfalls.md         # 反模式与真实踩坑记录 (§8)
├── templates/
│   ├── state.contract.json
│   ├── content.contract.json
│   ├── manifest.contract.json
│   ├── validate_content.py
│   └── smoke_test.gd
└── scripts/
    ├── prepare_cdn.mjs
    └── package_release.sh
```

判断：可以做成 Skill 包，但要分层。其中"可机械复刻"的部分是模板与脚本（契约格式、校验器、探针、发布流程）；"需判断"的部分是方法论（证据模型、后果制、机制转译）——后者应以决策树与反例形式写入，而不是伪代码。

7.3 五条反模式（看起来对但实际失效）

反模式	为什么看起来对	实际后果
用 CDN 分支引用托管可变产物	免账号、首屏快 30 倍	缓存不随提交更新 → 新旧文件混用 → 产物彻底打不开（§ 8.2）
把"进程被杀"归因于内存	确实同时存在内存紧张	真因是代码签名失效，误导排查方向数小时（§ 8.2）
只按名字查找交付物	之前一次成功过	漏掉同目录下另一个不同名的交付包，导致"素材没补齐"的误判（§ 8.2）
用关闭来隐藏不可用功能	界面看起来干净	若不同步说明，用户会以为按钮坏了；应显式下线并记录恢复开关（本项目的正确做法）
多 AI 并行改同一文件	效率高	一个 AI 会把另一个未完成的工作一起提交；应各自只提交自己负责的文件（§ 8.2）

8. 复盘：有效与失效

8.1 真正解决问题的

做法	证据
契约静态校验 + 五路径冒烟	在多次大改后仍保持 5/5，是"敢继续改"的底气
逐场景/逐说话人探针	一次性暴露"军中同伴没有立绘"这类只有真跑才看得见的问题

做法	证据
像素统计验收 UI	客观量化"看不清" (1.2:1 → 5.1:1) ，替代"感觉好点了吗"
分层证据模型	成为玩法的差异化亮点，而非普通分支叙事
消息协议强制附证据	让"AI 之间互相验收"从空谈变成可执行
私有素材仓库 + 哈希校验	清理 185MB 前先用 SHA-256 证明是重复数据，敢动手

8.2 走过的弯路（真实记录）

弯路一：CDN 提速导致产物彻底打不开

为把首屏从 12 分钟降到 20 秒，采用「HTML 留在 Pages + `<base>` 指向 jsDelivr + wasm 分三片重组」。链路当下可用，但jsDelivr 对分支引用（`@main`）的缓存不随提交更新；后续几次导出后，CDN 仍在发旧的 wasm 分片与旧资源包，页面却拿到新 HTML → 重组出的二进制与加载器不匹配 → 引擎无法实例化 → 游戏永远停在加载页。

教训：用可变引用指向 CDN 承载构建产物，必须每次更新后 purge，否则必然新旧混用。最终回退为自包含构建，并把优化重心放回体积本身（14.2MB → 8.0MB）。

弯路二：把"进程被杀"误判为内存不足

导出连续失败（exit 137），系统内存一度只剩 97MB，于是全组按内存方向排查（关应用、清进程）。释放到 1.2GB 后依旧被杀，最终在系统日志中找到真因：

```
amfid: Godot not valid: The signature on the file is invalid
kernel: code signature validation failed fatally
```

重新下载的 Godot 被 macOS AMFI 判定签名失效。`codesign --force --deep --sign -` 临时签名后立即恢复。

教训：exit 137 不等于 OOM。先看系统日志（AMFI/Jetsam），再谈资源。

弯路三：只按文件名找交付物

队长两次提示"素材补齐了"，我两次核查的是 `项目图片(1).zip` 并得出"没补"的结论；实际上补充件在另一个文件 `补充图片(1).zip`。

教训：核对交付应按清单与目录逐一比对，而不是按名字猜。

弯路四：多 AI 共用一个工作区

一个 AI 把另一个 AI 正在进行中的改动一并提交（内容无损，但作者归属混乱）。

教训：多智能体共用工作区时，各自只提交自己负责的文件。

8.3 教训清单

- 1. 客观证据 > 主观判断（像素、哈希、计数、日志）。
- 2. 归因必须先看系统层证据，再做资源层假设。

- 3. 交付核对接清单，不按名字。
- 4. 引入外部依赖（CDN、托管平台）前，先问"它什么时候会变、变了会怎样"。
- 5. 不可用功能要显式下线并留恢复开关，而非静默保留。
- 6. 多智能体的效率来自解耦（契约、边界、消息），而非并行度。

9. 结论

9.1 项目层面

《这真的是对的时间线吗？》第一章已经完成一次完整可复刻的垂直切片交付：三条发布通道、完整美术与音频、可运行的三结局、以及一套可重复执行的验证体系。首批美术需求全部交付接入，无遗留缺口。

9.2 方法论层面

本项目最有价值的产出不是代码，而是"多智能体如何协作交付一个真实产品"的可验证范式，其要点可以浓缩为三句：

- 1. 契约先行——让不同角色的工作可以在不互相破坏的前提下并行；
- 2. 证据说话——AI 之间无法凭"看起来对"互相验收，必须用可复现的测试与统计；
- 3. 边界清晰——角色、职责、文件、发布通道都要有明确归属。

9.3 可复刻性判断

结论：可以做成强大的 Skill 包，但正确形态是"模板 + 脚本 + 决策树 + 反模式"，而不是一份操作手册。

理由：本项目中被证明有效的部分，恰好分成两类——

- 可机械复刻的：三层契约格式、校验器、五路径冒烟、逐场景探针、像素验收、多端发布清单、资产分层治理。这些可以直接变成模板与脚本。
- 需要判断力的：把平台机制转译为机制、把失败写成后果、三层证据的层级划分、审校闭环中的分级与打回。这些应以决策树 + 真实反例的形式写入，因为它们依赖具体题材与团队。

而 § 8 的四条弯路，恰恰是最不该被省略的部分——它们记录的**不是**"怎么做对"，而是"什么做法看起来对但会失败"，这正是任何 Skill 包中最稀缺、也最难通过正向描述传递的知识。

附录：证据索引

类别	位置
公开代码仓库	github.com/MOHE007/is-this-the-right-timeline （71 次提交）
在线试玩	mohe007.github.io/is-this-the-right-timeline/
桌面发布	Release demo2-ch1-v0.2.4 （macOS 通用 / Windows 单文件）
私有素材仓库	github.com/MOHE007/ithrtt-assets （399 MB，已哈希校验）

类别	位置
产品说明计划书	docs/product-plan.md（初审送审稿 v2.0）
验证工具	godot/demo2/tools/（5 个）、web/prepare_cdn.mjs
实跑证据	godot/demo2/docs/run-2026-09-13/、run-2026-09-14-*（截图 + 日志）
协作记录	知识库 协作接口/（inbox 18 条消息、task_board、协议）
审校报告	docs/production/demo2_ch1_marvis_review_v01/v02/v021.md
契约族	docs/production/demo2_ch1_state_v02.json、content_v02_patch.json

本文由 DSH Desktop 依据项目全过程记录整理，所有结论均可通过上述证据索引复现。